

IoT Field Training

Simple esp32 task



Dana Natsheh
Cyber Robot
IoT



1. EXECUTIVE SUMMARY

This report covers the step-by-step setup completed on Day 1 of field training. The goal was to set up a laptop as a professional development station for embedded systems using Visual Studio Code. This involved installing the ESP32 code tools, fixing a major USB connection driver problem, writing code to blink an external LED on Pin 18, and uploading the project to GitHub.

2. SETTING UP THE TOOLS & VS CODE

Computers cannot run embedded C code directly because their built-in processors are different from the chip inside the ESP32. We need a special setup called a cross-compiler to package code on a computer so it can run on the ESP32 hardware.

Setup Steps

- 1.The Code Editor: We opened Visual Studio Code and installed the C/C++ Extension Pack to handle code highlighting and auto-fill tips.
- 2.The Framework: Since the old Microsoft "Embedded Tools" packet is no longer available, we installed the official ESP-IDF Extension by Espressif Systems.
- 3.The Installation Wizard: We used the "Express Setup" to automatically download the ESP32 tools, Python environments, and code builders directly from GitHub.

3. UNDERSTANDING THE PROJECT FOLDERS

Embedded projects cannot exist as just a single, isolated .c file. They need a specific folder structure so the code builder knows how to link everything together.

Folder Layout Breakdown

```
texttext
esp project/
├── .vscode/                # Hidden settings folder for the text editor
│   ├── c_cpp_properties.json # Tells VS where the ESP32 files live to clear red errors
│   ├── launch.json          # Settings used for advanced code debugging
│   └── settings.json        # Saves the specific tool paths for this project
├── build/                  # Temporary folder where compiled machine code is saved
├── main/                   # The folder where the developer writes code
│   ├── CMakeLists.txt      # Tells the builder what files to look at inside "main"
│   └── main.c              # Our main embedded C code file
└── CMakeLists.txt          # Master setup file that configures the entire project
Use code with caution.
```

Key Folder Details

- **Why the .vscode folder matters:** This folder clears out the red squiggly error lines under your #include statements. It tells VS Code exactly where the ESP32 library headers are saved on your computer.
- **Why the build folder is ignored:** When you compile your project, this folder generates thousands of temporary files. These files are huge and are not needed on GitHub, so we keep them hidden.
- **Why main.c uses app_main:** Standard computer programs look for a function called main(). However, the ESP32 runs a tiny operating system called FreeRTOS, which looks for a starting function called app_main() instead.





3. UNDERSTANDING THE PROJECT FOLDERS

The Blinking LED Code (Pin 18)

Here is the clean code i used to control the external LED connected to physical Pin 18 and print messages to the screen:

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"

#define BLINK_GPIO 18

static const char *TAG = "BlinkExample";

void app_main(void)
{
    gpio_reset_pin(BLINK_GPIO);

    gpio_set_direction(BLINK_GPIO, GPIO_MODE_OUTPUT);

    bool led_state = false;
    ESP_LOGI(TAG, "ESP32 Blink on Pin 18 Started!");

    while (1) {
        led_state = !led_state; // Toggle the state (True / False)

        gpio_set_level(BLINK_GPIO, led_state);

        if (led_state) {
            ESP_LOGI(TAG, "Pin 18 is HIGH (LED ON)");
        } else {
            ESP_LOGI(TAG, "Pin 18 is LOW (LED OFF)");
        }

        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```



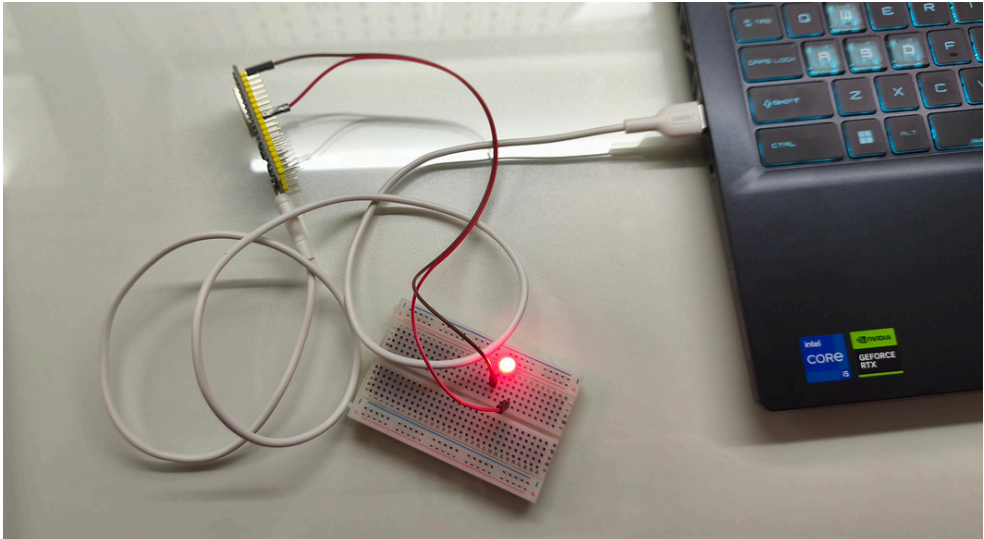


3. UNDERSTANDING THE PROJECT FOLDERS

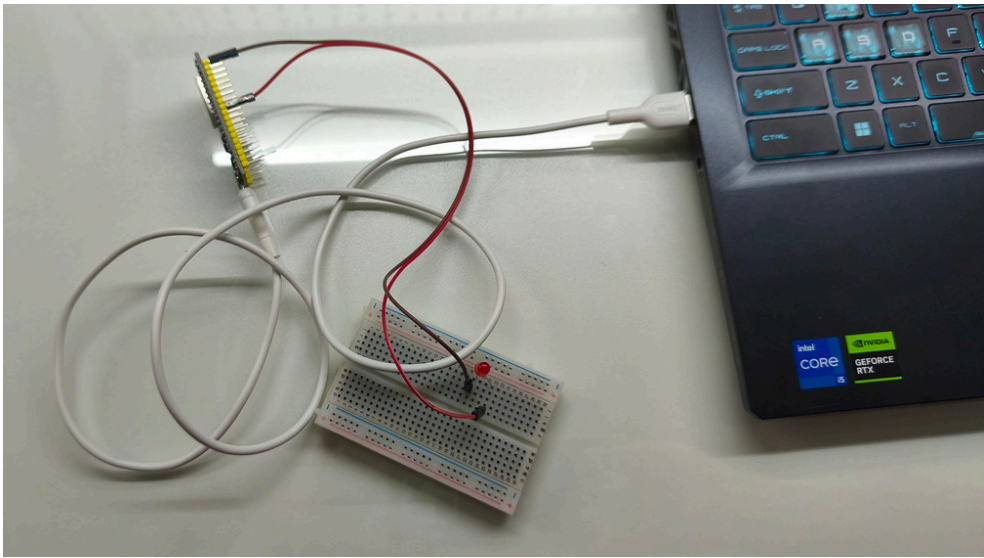
Sample Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  MEMORY  XRTOS  ...  
  
I (213269) BlinkExample: Pin 18 is LOW (LED OFF)  
I (214269) BlinkExample: Pin 18 is HIGH (LED ON)  
I (215269) BlinkExample: Pin 18 is LOW (LED OFF)  
I (216269) BlinkExample: Pin 18 is HIGH (LED ON)  
I (217269) BlinkExample: Pin 18 is LOW (LED OFF)  
I (218269) BlinkExample: Pin 18 is HIGH (LED ON)  
I (219269) BlinkExample: Pin 18 is LOW (LED OFF)  
I (220269) BlinkExample: Pin 18 is HIGH (LED ON)  
I (221269) BlinkExample: Pin 18 is LOW (LED OFF)  
I (222269) BlinkExample: Pin 18 is HIGH (LED ON)  
I (223269) BlinkExample: Pin 18 is LOW (LED OFF)  
I (224269) BlinkExample: Pin 18 is HIGH (LED ON)  
I (225269) BlinkExample: Pin 18 is LOW (LED OFF)  
I (226269) BlinkExample: Pin 18 is HIGH (LED ON)
```

Serial Monitor Output



LED ON



LED OFF





4. SOLVING PROBLEMS ENCOUNTERED

Problem 1: Missing Git Software

The tool installation crashed at the beginning because Git was not installed on the computer. The ESP32 installer needs Git to download code templates from the internet.

- Solution: We downloaded and installed Git for Windows from its official website, which allowed the ESP32 setup manager to finish without errors. [1]

Problem 2: "No Serial Ports Found" (Missing Drivers)

When the ESP32 board was plugged into the laptop via a USB cable, VS Code could not find it, and it did not show up in the computer's Device Manager.

The Cause

Most ESP32 development boards use a small bridge chip (like the CP210x or CH340) to let the computer speak to the microchip over USB. The official documentation website didn't install this automatically. Without this driver software, Windows cannot recognize the board.

The Solution

1. We manually downloaded the CP210x USB to UART Bridge Driver files from another site.
2. We right-clicked the installation file named silabser.inf and selected Install.
3. Windows immediately recognized the board and assigned it to COM3, letting us flash our code successfully.

5. SAVING CODE TO GITHUB

To save my hard work and track future training assignments, I linked the local folders to a GitHub repository named Dana_IoT.

Using a .gitignore File

To avoid uploading hundreds of megabytes of heavy, temporary computer-generated files from our build folder, we created a file named .gitignore in our project root directory and added these two lines:

```
text
build/
.vscode/
Use code with caution.
```

Github commands to upload code:

```
git init
git remote add origin https://github.com
git pull origin main
git add .
git commit -m "Add initial ESP32 blink LED project folder"
git push -u origin main
```

The files are now safely hosted online under GitHub profile!





6. ALTERNATIVE TOOL OPTION: PLATFORMIO VS. ESP-IDF

While i used the official ESP-IDF extension, another popular option in VS Code is PlatformIO.

Feature	Official ESP-IDF Extension (Used)	PlatformIO IDE Extension
Target Chips	Works only with Espressif chips (ESP32).	Works with almost any chip (Arduino, STM32, ESP32).
Setup Difficulty	Requires code folder structures and CMake files.	Uses one single settings file (platformio.ini).
Learning Value	High. Forces you to learn professional engineering methods.	Medium. Hides the background setup details from you.

CONCLUSION

The official ESP-IDF path we set up today is the best choice for professional training. It teaches you how real-world embedded software systems compile, handle files, and interface directly with hardware chips.

What I Learned on Day 1

- **Setting Up the Tools:** I learned how to install the special ESP-IDF tools inside VS Code so my computer can write code for the ESP32 chip.
- **Project Folders:** I learned how an embedded project folder is structured and why we need separate files to link the hardware libraries together.
- **Microcontroller Basics:** I learned that the ESP32 looks for a starting function called app_main() instead of the standard computer main().
- **Controlling Hardware:** I learned how to set up a physical pin (Pin 18) as an output to send electricity and blink an external LED.
- **Fixing Driver Problems:** I learned how to install missing USB drivers manually so my computer can find the board on COM3.

